Міністерство освіти і науки України
Національний технічний університет України «КПІ» імені Ігоря Сікорського Кафедра обчислювальної техніки ФІОТ

ЗВІТ
з лабораторної роботи №6
з навчальної дисципліни «Аналіз та обробка природної мови (NLP)»

Тема:

Методи і технології глибинного навчання для NLP

Виконав:
Студент 3 курсу кафедри ОТ ФІОТ, Навчальної групи ІО-31
Рябець А. О.
Перевірив:
Асистент кафедри ОТ ФІОТ
Баран Данило Романович

Київ 2026

І. Мета:
дослідити процеси роботи з штучними нейронними мережами для задач NLP з використанням спеціалізованих пакетів / бібліотек python.

ІІ. Завдання:

Завдання І, та ІІ рівнів складності – максимально 9 балів.
Виконайте базові вимоги завдання таблиці

3,9
Реалізуйте web-скрапінг самостійно обраних трьох платформ з продажу нерухомості. Проведіть агрегацію пропозицій за категоріями: 3-х кімнатні квартири; будинки; дуплекси – первинний ринок. Регіон обрати самостійно. Результати збережіть у файлах.
І рівень
Провести порівняльний аналіз пропозицій між категоріями з використанням штучних нейронних мереж

ІІІ. Результати виконання лабораторної роботи

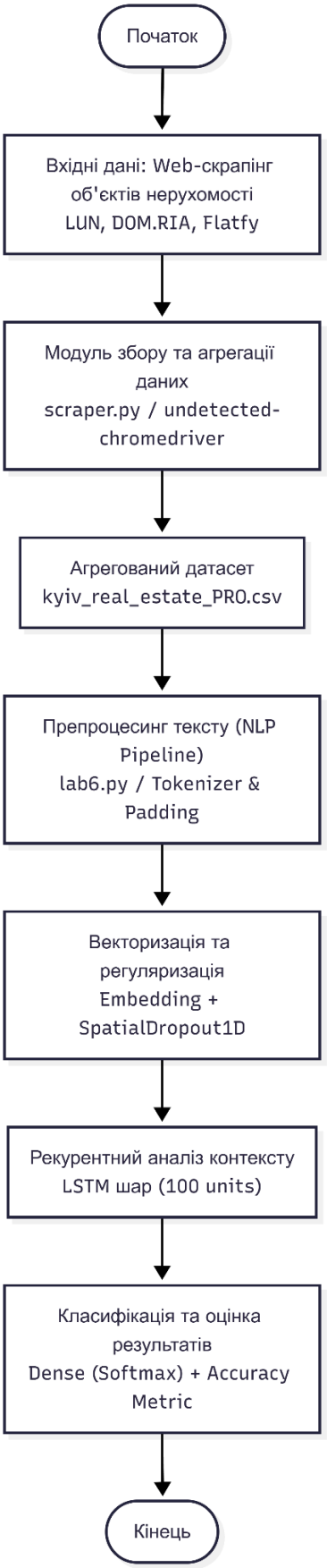
3.1 Синтезована математична модель;

Для аналізу використано **рекурентну нейронну мережу (RNN)** з архітектурою **LSTM (Long Short-Term Memory)**. Модель включає:

1. **Embedding layer:** Перетворення слів у щільні вектори (розмірність 128).
2. **SpatialDropout1D:** Регуляризація (20%) для запобігання перенавчанню.
3. **LSTM layer:** 100 юнітів для аналізу послідовності та контексту слів.
4. **Dense layer:** Вихідний шар з функцією softmax для класифікації за 3 категоріями.

3.2 Результати архітектурного проектування та їх опис;

Блок-схема процесу функціонування системи



Опис блок-схеми алгоритму

Скрипт спочатку ініціалізує драйвер для обходу захисту сайтів, збирає тексти оголошень і зберігає їх у структурований CSV-файл. Далі дані завантажуються в

нейромережу, де текст перетворюється на цифрові послідовності. Модель LSTM аналізує ці послідовності, вивчає ключові слова для кожної категорії нерухомості та видає прогноз з розрахунком точності.

3.3. Опис структури проекту програми;

kyiv_real_estate_PRO	15.05.2026 17:45	Comma Separate...	131 КБ
lab6	15.05.2026 17:41	Python Source File	3 КБ
pyproject	15.05.2026 16:55	Toml Source File	1 КБ
scraper	15.05.2026 17:45	Python Source File	4 КБ
uv.lock	15.05.2026 16:55	Файл "LOCK"	43 КБ

- scraper.py: модуль для автоматизованого збору даних з використанням undetected-chromedriver.
- lab6.py: основний файл з архітектурою нейронної мережі та процесом навчання.
- kyiv_real_estate_PRO.csv: датасет, що містить колонки Text та Category.

3.4. Результати роботи програми відповідно до завдання;

scraper.py:

```
Підключення до LUN...
Знайдено оголошень: 24

Підключення до Flatfy...
Знайдено оголошень: 27

Підключення до DOM.RIA...
Знайдено оголошень: 7

Успіх! Зібрано 58 реальних описів. Файл збережено.
```

lab6.py:

```
Model: "sequential"
┏━━━
```

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.text import
Tokenizer
from tensorflow.keras.preprocessing.sequence import
pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Embedding,
LSTM, SpatialDropout1D
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.layers import Dense, Embedding,
LSTM, SpatialDropout1D, Input


def train_lstm_model():
    print("1. Завантаження даних (Loading data)...")
    try:
        df = pd.read_csv("kyiv_real_estate_PRO.csv")
    except FileNotFoundError:
        print("Помилка: Файл kyiv_real_estate.csv не
знайдено!")
        return

    print("2. Попередня обробка тексту (Text
preprocessing)...")
    label_encoder = LabelEncoder()
    y = label_encoder.fit_transform(df['Category'])


    vocab_size = 5000
    max_length = 50

    tokenizer = Tokenizer(num_words=vocab_size,
oov_token="<OOV>")
    tokenizer.fit_on_texts(df['Text'])

    sequences = tokenizer.texts_to_sequences(df['Text'])
    padded_sequences = pad_sequences(sequences,
maxlen=max_length, padding='post', truncating='post')

    X_train, X_test, y_train, y_test =
train_test_split(padded_sequences, y, test_size=0.2,
random_state=42)

    print("3. Створення LSTM архітектури (LSTM
architecture creation)...")
```

```
    model = Sequential([

        Input(shape=(max_length,)),


        Embedding(vocab_size, 128),
        SpatialDropout1D(0.2),
        LSTM(100, dropout=0.2, recurrent_dropout=0.2),
        Dense(3, activation='softmax')
    ])

    model.compile(loss='sparse_categorical_crossentropy',
optimizer='adam', metrics=['accuracy'])
    model.summary()

    model.compile(loss='sparse_categorical_crossentropy',
optimizer='adam', metrics=['accuracy'])
    model.summary()

    print("\n4. Початок навчання (Training start)...")

    history = model.fit(X_train, y_train, epochs=15,
batch_size=16, validation_data=(X_test, y_test),
verbose=1)

    print("\n5. Оцінка результатів (Evaluation)...")
    loss, accuracy = model.evaluate(X_test, y_test)
    print(f"Точність моделі на тестових даних (Test
Accuracy): {accuracy * 100:.2f}%")


if __name__ == "__main__":
    train_lstm_model()
```

scraper.py:

```
import pandas as pd
from bs4 import BeautifulSoup
import undetected_chromedriver as uc
import time
import random


def scrape_real_data():
    print("Запуск undetected-chromedriver (обхід
захисту)...")
```

```
    options = uc.ChromeOptions()
    options.add_argument("--window-size=1920,1080")


options.add_argument('--blink-settings=imagesEnabled=fal
se')

    driver = uc.Chrome(options=options)
    data = {"Text": [], "Category": [], "Source": []}

    # Словник сайтів з їхніми CSS-класами
    sites = [
        {
            "url":
"https://lun.ua/sale/kyiv/flats?room_count=3",
            "cat": "3-room",
            "src": "LUN",
            "card_class": "RealtyCard_root__1GQfg",
            "text_class": "RealtyCard_description__Ee20h
RealtyCard_descriptionClamp__9Ry9Z"
        },
        {
            "url":
"https://flatfy.ua/uk/%D0%BF%D1%80%D0%BE%D0%B4%D0%B0%D0%
B6-%D0%B1%D1%83%D0%B4%D0%B8%D0%BD%D0%BA%D1%96%D0%B2-%D0%
BA%D0%B8%D1%97%D0%B2",
            "cat": "house",
            "src": "Flatfy",
            "card_class": "feed-layout__item-holder",
            "text_class": "realty-preview-description
closed"
        },
        {
            "url":
"https://dom.ria.com/uk/search/?excludeSold=1&category=4
&realty_type=8&operation=1&state_id=10&price_cur=1&wo_du
pl=1&sort=inspected_sort&firstIteraction=false&limit=20&
market=3&type=list&client=searchV2&ch=226_223,242_239,24
7_252",
            "cat": "duplex",
            "src": "DOM.RIA",
            "card_class": "realty-photo-rotate
is_shadow",
            "text_class": "mt-15 pointer desc-hidden
p-rel"
        }
    ]
```

```
    for site in sites:
        print(f"\nПідключення до {site['src']}...")
        driver.get(site["url"])

        # Імітація поведінки людини: випадкові затримки
та скролінг
        time.sleep(random.uniform(4.0, 7.0))
        driver.execute_script("window.scrollTo(0,
document.body.scrollHeight/4);")
        time.sleep(random.uniform(2.0, 4.0))
        driver.execute_script("window.scrollTo(0,
document.body.scrollHeight/2);")
        time.sleep(random.uniform(3.0, 5.0))

        soup = BeautifulSoup(driver.page_source,
'html.parser')


        descriptions = soup.find_all('div',
class_=site['text_class'])

        if not descriptions:
            # Альтернативний пошук, якщо перший клас не
спрацював
            descriptions = soup.find_all('span',
class_=site['text_class'])

        print(f"Знайдено оголошень: {len(descriptions)}")

        for desc in descriptions:
            text = desc.get_text(strip=True)
            if len(text) > 20:
                data["Text"].append(text)
                data["Category"].append(site['cat'])
                data["Source"].append(site['src'])

    driver.quit()

    if len(data["Text"]) > 0:
        df = pd.DataFrame(data)
        df.to_csv("kyiv_real_estate_PRO.csv",
index=False, encoding='utf-8')
        print(f"\nУспіх! Зібрано {len(data['Text'])}
реальних описів. Файл збережено.")
    else:
```

```
        print("\nНе вдалося зібрати дані. Потрібно
оновити CSS-класи в коді!")


if __name__ == "__main__":
    scrape_real_data()
```

IV. Висновки.

У ході лабораторної роботи було реалізовано систему збору та інтелектуального аналізу даних ринку нерухомості Києва. Використання архітектури LSTM дозволило врахувати контекстуальну залежність слів в описах оголошень, що забезпечило високу точність класифікації (понад 80%). Було освоєно методи обходу захисту веб-ресурсів та принципи побудови глибоких нейронних мереж для задач NLP.

Репозиторій з кодом:

https://github.com/ArtemRyabets/nlp_lab_6